Multi Vendor Earning Blog Site with AI

M. Shahzaib Karamat **F22-1580**

M. Akhlaq Raza **F22-1609**

Abdullah Farooq **F22-1171**



March 2026

Department of Computer Science

Lahore Leads University

Lahore – PAKISTAN

Submission Form for Final-Year

PROJECT REPORT

PROJECT ID	52	NUMBER OF MEMBERS	3

TITLE	Multi Vendor Earning Blog Site with AI

SUPERVISOR NAME	Miss. Tuba Gull

MEMBER NAME	REG. NO.	EMAIL ADDRESS
M. Shahzaib Karamat	F22-1580	shahzaibkaramat007@gmail.com
M. Akhlaq Raza	F22-1609	hafizakhlaq77@gmail.com
Abdullah Farooq	F22-1171	abdullahfarooq56789@gmail.com

CHECKLIST:

Number of pages in this report

I/We have enclosed the soft-copy of this document along-with the codes and scripts created by myself/ourselves — YES / NO

My/Our supervisor has attested the attached document — YES / NO

MEMBERS' SIGNATURES

_____

_____

_____

Supervisor's Signature

Note 1: This paper must be signed by your supervisor

Note 2: The soft-copies of your project report, source codes, schematics, and executables should be delivered in a CD/DVD

This work, entitled "Multi Vendor Earning Blog Site with AI" has been approved for the award of

Bachelors of Science in Computer Science

Date 12 March 2026

External Examiner: _____

FYP Coordinator: _____

Head of Department: _____

Department of Computer Science

LAHORE LEADS UNIVERSITY

LAHORE – PAKISTAN

Declaration

"No portion of the work referred to in the dissertation has been submitted in support of an application for another degree or qualification of this or any other university/institute or other institution of learning".

MEMBERS' SIGNATURES
_____
_____
_____

Acknowledgements

In the name of God, the most kind and most merciful

I would like to thank Abdullah who kept backing me up in all the times, both financially and morally...

I would also like to thank Akhlaq Raza for his guidance and encouraging me to work hard and smart. I have found him very helpful while discussing the optimization issues in this dissertation work. His critical comments on my work have certainly made me think of new ideas and techniques in the fields of optimization and software simulation.

I am grateful to the God Almighty who provides all the resources of every kind to us, so that we make their proper use for the benefit of mankind. May He keep providing us with all the resources, and the guidance to keep helping the humanity.

Abstract

This project presents a modern blog platform integrated with AI-powered content moderation to ensure safe and responsible publishing. Unlike traditional AI-based systems that generate content, this platform focuses on analyzing user-submitted content in real time to detect harmful, abusive, or policy-violating material. The AI system evaluates content before publication and prevents uploads that may negatively impact individuals or communities, while allowing safe and appropriate content to be published seamlessly.

In addition to content creation, the platform provides users with monetization opportunities similar to Blogger. Users can earn revenue by displaying advertisements on their blogs through integration with Google AdSense. This enables content creators to generate income while maintaining high-quality and policy-compliant content standards enforced by the AI moderation system.

The system includes secure user authentication, a personalized dashboard, and a structured content management interface. It is built using modern web technologies such as Node.js, Express.js, MongoDB, and EJS, ensuring scalability, performance, and maintainability. By combining user freedom, monetization capabilities, and intelligent safety checks, the platform aims to create a trustworthy and sustainable digital environment for content creators.

Table of contents

CHAPTER 1 Introduction & Project Overview **8**

1.1 What is the Blog Website? 8

1.2 The Multi-Vendor Concept 8

1.3 Project Goals & Objectives 8

1.4 Technology Stack 9

1.5 Team & University Details 9

CHAPTER 2 System Architecture **10**

2.1 High-Level Architecture 10

2.2 MVC Pattern Deep Dive 10

2.3 Project Folder Structure 10

2.4 Middleware Architecture 11

2.5 Express Routing System 11

CHAPTER 3 Authentication & Security **12**

3.1 Authentication System 12

3.2 JWT & Refresh Token System 13

3.3 Session Management 13

3.4 Role-Based Access Control (RBAC) 13

3.5 Content Security Policy (CSP) 14

3.6 API Security 14

3.7 Input Validation Rules 14

CHAPTER 4 Admin Features **15**

4.1 Admin Dashboard 15

4.2 Category Management 15

4.3 Topic Management 16

4.4 Rate Configuration 16

4.5 Category Status Control 16

CHAPTER 5 Frontend & User Experience **17**

5.1 EJS Template Engine 17
5.2 Dynamic Topic Rendering 17
5.3 Navbar & Layout Components 18
5.4 Flash Messaging System 18
5.5 SEO Friendly Routing 18

CHAPTER 6 Content Moderation & Forms 19

6.1 Content Moderation AI 19
6.2 Form Handling 19
6.3 Async Error Handling 20
6.4 Error Handling System 20

CHAPTER 7 Data Models & Database 21

7.1 User Model 21
7.2 Category Model 22
7.3 Topic Model 22
7.4 Session Storage 23
7.5 Environment Configuration 23

CHAPTER 8 Complete Feature Reference 25

8.1 All 45 Features 25
8.2 Feature Summary by Module 28

Chapter 1

1 Introduction & Project Overview

This chapter introduces the Blog Website project — a multi-vendor earning platform built as a Final Year Project at Lahore Leads University. It covers the vision, goals, and the technology stack that powers the platform.

1.1 What is the Blog Website?

The Blog Website is a multi-vendor blogging platform that allows multiple users to create profiles, publish blog articles across various categories, and earn money based on the number of views their content receives. Unlike a single-author blog, this platform is designed to support many writers simultaneously, each managing their own content independently.

The platform supports two distinct user roles: **Admin** and **Normal User**. Administrators have full control over categories, topics, rate configurations, and site-wide settings. Normal users can register, log in, create blog posts, and track their earnings through a personal dashboard.

Key Concept	Writers earn based on a configurable rate-per-1000-views model. Admins set the earning rate per category, so high-traffic categories may offer different earning rates than niche ones.

1.2 The Multi-Vendor Concept

The term multi-vendor in this context refers to the platform supporting many independent content creators simultaneously, each acting as an independent 'vendor' of content. This is fundamentally different from a company blog where a single editorial team publishes content. Key characteristics include:

- Any registered user can become a content creator (writer)

• Each writer manages their own articles independently

• Writers earn based on individual content performance

• Admins set the earning rules but do not interfere with writer content (beyond moderation)

• The platform scales naturally as more writers join

1.3 Project Goals & Objectives

The Blog Website was conceived with the following strategic goals:

Goal 1: Build a Real-World Earning Platform

Create a production-ready application simulating how content monetization systems work. The rate-per-1000-views model is used by major platforms and gives writers a transparent earning structure.

Goal 2: Implement Enterprise-Grade Authentication

Use JWT with refresh tokens — the same authentication pattern used by companies like Google and Facebook — to provide secure, stateless authentication that scales.

Goal 3: Demonstrate Role-Based Access Control

Build a system where different users have clearly defined permission boundaries, demonstrating understanding of authorization vs authentication concepts.

Goal 4: Integrate AI Content Safety

Integrating a moderation API demonstrates understanding of third-party API integration and content policy enforcement at the application layer.

Goal 5: Apply Full-Stack MVC Architecture

Build a complete, organized codebase using the Model-View-Controller pattern — the industry-standard approach for separating concerns in web applications.

1.4 Technology Stack

Every technology in the stack was chosen deliberately:

Layer	Technology	Purpose
Runtime	Node.js	Server-side JavaScript execution environment
Framework	Express.js	HTTP routing, middleware, and MVC architecture
Template Engine	EJS	Server-side dynamic HTML rendering
Database	MongoDB	NoSQL document storage for users, posts, sessions
ODM	Mongoose	MongoDB schema modeling and validation
Auth	JWT + bcrypt	Token-based auth and password hashing
Sessions	express-session + connect-mongo	Persistent sessions in MongoDB
Styling	Tailwind CSS	Utility-first responsive CSS framework
Security	Helmet / CSP	HTTP security headers and Content Security Policy
AI	Moderation API	Harmful content detection before database writes

1.5 Team & University Details

Field	Details
University	Lahore Leads University
Department	BSCS Bachelor Study In Computer Science
Project Type	Final Year Project (FYP)
Academic Year	2022 / 2026
Team Member 1	M. Shahzaib Karamat F22-1580
Team Member 2	M. Akhlaq Raza F22-1609
Team Member 3	Abdullah Farooq F22-1171

Chapter 2

2 System Architecture

This chapter describes how the Blog Website is structured internally — from the overall backend organization down to the database models, middleware pipeline, and request lifecycle.

2.1 High-Level Architecture

At the highest level, the Blog Website follows the classic web application architecture: a client (browser) communicates with a server (Express.js on Node.js) which reads and writes data to a database (MongoDB). The system uses a monolithic architecture where both the API and frontend are served by the same Express application, making deployment significantly simpler.

2.2 MVC Pattern Deep Dive

The Model-View-Controller (MVC) pattern is the architectural backbone of the entire application. Every feature coordinates three components:

Component	Responsibility	Blog Website Example
Model	Defines data structure, validation rules, database queries	User.findOne({email}) — finds user in MongoDB
View	Renders HTML output using data from controller	home.ejs renders article cards from controller data
Controller	Receives request, calls Model, sends data to View	articleController.create() — validates, saves, redirects

2.3 Project Folder Structure

Path	Contents	Responsibility
/controllers	articleController.js, authController.js	Business logic per feature domain
/routes	authRoutes.js, articleRoutes.js, adminRoutes.js	URL mapping and route definitions
/models	User.js, Category.js, Topic.js	Mongoose schemas and model exports
/middleware	authMiddleware.js, adminGuard.js	Reusable middleware functions
/views	/partials, /auth, /admin, /articles	EJS templates organized by feature
/public	/css, /js, /images	Static assets served by Express
/utils	asyncHandler.js, tokenUtils.js	Shared helper functions
.env	Not committed to Git	All secrets and configuration

2.4 Middleware Architecture

Middleware functions sit between the incoming HTTP request and the controller handler, forming a pipeline that processes, validates, and guards routes:

- Authentication Middleware — verifies JWT token, attaches user to req.user
- Role Guard Middleware — checks req.user.role for admin-only routes
- Validation Middleware — checks required fields, data types before processing
- Flash Middleware — attaches success/error messages for EJS rendering
- Async Error Handler — catches rejected promises without try/catch boilerplate
- Session Middleware — initializes express-session with MongoDB store

2.5 Express Routing System

Route File	Base Path	Purpose
authRoutes.js	/auth	Login, register, logout, password change
articleRoutes.js	/articles	Create, read, update, delete blog posts
adminRoutes.js	/admin	Category, topic, rate management
profileRoutes.js	/profile	User profile and dashboard routes
indexRoutes.js	/	Home, search, category browse pages

Chapter 3

3 Authentication & Security

Security is a core concern of the platform. This chapter covers every layer of authentication and security implemented, from JWT tokens and session handling to CSP headers and API protection.

3.1 Authentication System

The authentication system provides a complete lifecycle for user identity management. It handles registration, login, logout, and password changes through dedicated routes and controller methods.

Step	Action	Technical Detail
1	Registration	User submits email + password. Password hashed with bcrypt (10 salt rounds). User document created in MongoDB.
2	Login	Credentials verified against hashed password. On success, access JWT and refresh JWT generated and set as HTTP-only cookies.
3	Route Access	Auth middleware extracts JWT from cookie, verifies signature, attaches decoded user to req.user.
4	Refresh	When access token expires, refresh token verified from DB to issue a new access token silently.
5	Logout	Refresh token deleted from DB. Cookies cleared. Session destroyed. User redirected to login.
6	Pwd Change	Old password verified, new password hashed, user document updated.

3.2 JWT & Refresh Token System

JSON Web Tokens (JWT) are the core of stateless authentication. The platform uses a dual-token system — a short-lived access token and a long-lived refresh token.

Access Token	Short-lived (15 minutes). Sent as HTTP-only cookie. Verified on every protected request. Contains user ID and role in the signed payload.
Refresh Token	Long-lived (7 days). Stored in the users collection in MongoDB. Used only to generate new access tokens. Invalidated on logout to prevent reuse.

3.3 Session Management

In addition to JWT cookies, the platform uses express-session backed by connect-mongo to maintain server-side sessions stored in MongoDB. This provides an additional layer of state management for flash messages and UI personalization.

3.4 Role-Based Access Control (RBAC)

Permission	Admin	Normal User
Create Categories	YES	NO
Create Topics	YES	NO
Set Earning Rates	YES	NO
Manage Category Status	YES	NO
Publish Blog Articles	YES	YES
View Own Dashboard	YES	YES
Change Own Password	YES	YES
Access Admin Panel	YES	NO

3.5 Content Security Policy (CSP)

CSP headers tell browsers which sources of content are trusted, preventing XSS attacks:

• script-src restricted to trusted CDN domains only (e.g., cdnjs.cloudflare.com)

• Unsafe inline scripts blocked entirely to prevent XSS injection

• Network requests restricted to same-origin and approved external APIs

• Object, frame, and embed sources disabled to prevent clickjacking

3.6 API Security

All API endpoints are protected by authentication middleware that verifies tokens before allowing access. Input validation middleware ensures all incoming data is checked for correct format, required fields, and data type integrity before reaching controllers.

3.7 Input Validation Rules

Field	Validation Rules
username	Required, 3–30 chars, alphanumeric + underscores, unique in DB
email	Required, valid RFC 5322 email format, unique in DB
password	Required, minimum 8 characters, at least 1 number
confirmPassword	Must exactly match password field
category name	Required, 2–50 chars, trimmed of whitespace
rate_per_1000	Required, numeric, positive number, max 100
article title	Required, 10–200 characters
article body	Required, minimum 100 characters

Chapter 4

4 Admin Features

The Admin Dashboard is the control center of the platform. This chapter documents all administrative capabilities including category management, topic organization, earning rate configuration, and content status control.

4.1 Admin Dashboard

The admin panel is accessible only to users with role='admin'. It provides a centralized interface for managing all aspects of the platform:

- Overview statistics — total users, articles, categories, and active topics
- Category creation form with name, rate, and status fields
- Topic creation form with category assignment
- Category list with edit, enable/disable, and delete actions
- Topic list with category filter and bulk operations
- Rate management — update rate_per_1000 for any category at any time

4.2 Category Management

Categories are the primary organizational units. Admins have full control to create unlimited categories — there is no hard limit. Each category carries a configurable earning rate and can be activated or deactivated at any time.

Category Schema	name: String (required, unique) | status: String ['active', 'inactive'] | rate_per_1000: Number (earnings per 1,000 article views) | createdAt: Date

4.3 Topic Management

Topics are sub-groupings within categories allowing writers to tag articles more specifically. Each topic is linked to exactly one category via a MongoDB ObjectId reference.

Topic Schema	name: String (required) | categoryId: ObjectId (ref: Category, required) | createdAt: Date

4.4 Rate Configuration

The earning system uses a rate_per_1000 model. For every 1,000 views an article receives, the writer earns the configured rate for that article's category:

Category	rate_per_1000 ($)	100K Views Earns	500K Views Earns
Technology	$1.20	$120.00	$600.00
Business	$1.50	$150.00	$750.00
Health	$0.90	$90.00	$450.00
Lifestyle	$0.60	$60.00	$300.00

4.5 Category Status Control

Status	Writer Can Select	Existing Articles	Earning Continues
active	YES	Visible and accessible	YES
inactive	NO	Remain published	YES (existing articles)

Chapter 5

5 Frontend & User Experience

This chapter covers the frontend layer — how data flows from the backend to rendered HTML, how users interact with the interface, and how the platform maintains a responsive, dynamic, and consistent UI.

5.1 EJS Template Engine

EJS (Embedded JavaScript Templates) is used for server-side rendering. Controllers pass data objects to EJS views which inject them into HTML at render time. This ensures every page is fully rendered on the server before reaching the browser, benefiting SEO.

Syntax	Purpose	Example
<%= value %>	Output escaped HTML value	<%= user.username %> → shahzaib
<%- html %>	Output raw unescaped HTML	<%- flashMessage %> for HTML messages
<% code %>	Execute JS logic	<% if(user.role==='admin') { %>
<% for(...) %>	Loop over arrays	<% articles.forEach(a => { %>
<%- include('path') %>	Include partial template	<%- include('partials/navbar') %>

5.2 Dynamic Topic Rendering

When a writer selects a category, the topic dropdown instantly updates to show only topics for that category — without a page reload. This is implemented using data attributes:

```
<select                                    data-rpm-groups='<%-
JSON.stringify(topicGroups) %>'>
```

Performance Benefit	By embedding data in the initial HTML response, the page has everything it needs from the start. Zero extra HTTP requests means faster perceived performance and simpler client code.

5.3 Navbar & Layout Components

The navbar dynamically switches between guest state (login/register buttons) and authenticated state (user menu, dashboard link, logout). This is controlled server-side in EJS using res.locals.currentUser.

5.4 Flash Messaging System

Flash messages provide immediate contextual feedback after user actions. They are stored in the session and cleared after display, appearing only once.

- success — green notification for registration successful, article published, password changed
- error — red notification for login failed, validation error, content flagged by AI
- info — blue notification for informational messages
- warning — orange notification for session expiry warnings

5.5 SEO Friendly Routing

Page	URL Pattern	SEO Benefit
Article	/articles/how-to-learn-nodejs	Contains keywords, human readable
Category	/category/technology	Describes the content type
Author	/writers/shahzaib	Personal brand and backlink value
Search	/search?q=mongodb	Clean query parameter format

Chapter 6

6 Content Moderation & Forms

Keeping the platform safe requires both automated content screening and robust form validation. This chapter documents the AI moderation integration, form handling patterns, and the error management system.

6.1 Content Moderation AI

Before any user-submitted content is saved to the database, it is passed through a third-party AI Moderation API. This API analyzes the text and returns a classification indicating whether the content is safe, potentially harmful, or explicitly toxic.

Stage	Description
1. Submission	User submits article text through the blog creation form
2. API Call	Backend sends content to the moderation API endpoint before any DB write
3. Classification	API returns a label: SAFE / POTENTIALLY_HARMFUL / TOXIC
4. Decision	SAFE content proceeds to save. Harmful content is blocked with error message
5. Feedback	User sees a flash error explaining the content was flagged and must be revised

6.2 Form Handling

The Express server processes all form submissions through dedicated controller methods. Each form has its own validation middleware and controller:

Form	Route	Controller Method
Login	POST /auth/login	authController.login
Register	POST /auth/register	authController.register
Change Password	POST /auth/change-password	authController.changePassword
Create Article	POST /articles	articleController.create
Create Category	POST /admin/categories	adminController.createCategory
Create Topic	POST /admin/topics	adminController.createTopic

6.3 Async Error Handling

All controller functions are wrapped in an asyncHandler utility that automatically catches any rejected promises and forwards them to the Express error handling middleware:

asyncHandler Pattern	const asyncHandler = fn => (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next); — Any thrown error or rejected promise is caught and passed to next(err), triggering the global error handler middleware.

6.4 Error Handling System

HTTP Status	Type	Description
400 Bad Request	Validation Error	Missing fields, wrong formats, duplicate email
401 Unauthorized	Auth Failure	No token, invalid token, expired token
403 Forbidden	Authorization	Authenticated but insufficient role/permissions
404 Not Found	Not Found	Article, user, or category doesn't exist
409 Conflict	Duplicate	Username or email already registered

HTTP Status	Type	Description
500 Server Error	System Error	Unexpected error — logged, generic message to user

Chapter 7

7 Data Models & Database

This chapter provides complete documentation of all MongoDB data models, their fields, validation rules, and relationships — plus the environment configuration system.

7.1 User Model

Field	Type	Constraints	Purpose
_id	ObjectId	Auto-generated	Unique document identifier
username	String	required, unique, trim	Public display name
email	String	required, unique, lowercase	Login identifier
password	String	required, minlength:8	bcrypt hash — never plaintext
role	String	admin | user (default: user)	Controls access level
refreshToken	String	optional	Current JWT refresh token
totalEarnings	Number	default:0	Cumulative earning balance
createdAt	Date	auto-generated	Account creation timestamp

Password Security	The User model NEVER stores plaintext passwords. The Mongoose pre-save hook automatically hashes the password: userSchema.pre('save', async function() { if(this.isModified('password')) this.password = await bcrypt.hash(this.password, 10); });

7.2 Category Model

Field	Type	Constraints	Purpose
_id	ObjectId	Auto-generated	Unique identifier
name	String	required, unique, trim	Category display name
status	String	enum:['active','inactive']	Visibility control
rate_per_1000	Number	required, min:0	USD earnings per 1K views
description	String	maxlength:200, optional	Category description
createdAt	Date	auto	Creation timestamp

7.3 Topic Model

Field	Type	Constraints	Purpose
_id	ObjectId	Auto-generated	Unique identifier
name	String	required, trim	Topic display name
categoryId	ObjectId	ref:'Category', required	Parent category reference
articleCount	Number	default:0	Articles under topic
createdAt	Date	auto	Creation timestamp

7.4 Session Storage

Sessions are stored in MongoDB using connect-mongo as the session store adapter for express-session. Sessions persist across server restarts and scale to multi-process deployments.

Session Config	secret: SESSION_SECRET | resave: false | saveUninitialized: false | store: MongoStore.create({mongoUrl: MONGO_URI}) | cookie.secure: true | cookie.httpOnly: true

7.5 Environment Configuration

Variable	Purpose	Security Level
MONGO_URI	Full MongoDB connection string including credentials	Critical
JWT_SECRET	Secret key for signing access tokens	Critical
JWT_REFRESH_SECRET	Separate secret for refresh token signing	Critical
SESSION_SECRET	Secret for express-session cookie signing	High
MODERATION_API_KEY	API key for the content moderation service	Medium
PORT	HTTP server port (default 3000)	Low
NODE_ENV	'development' or 'production' — controls error verbosity	Medium

Chapter 8

8 Complete Feature Reference

This chapter provides a complete indexed reference of all 45 documented features of the Blog Website platform, with full descriptions.

8.1 All 45 Features

#	Feature	Description
1	Authentication System	Complete user auth lifecycle — register, login, logout, password change, protected routes.
2	JWT Authentication	JSON Web Tokens for stateless user verification on all protected backend routes.
3	Refresh Token System	Long-lived refresh tokens stored in MongoDB for seamless, secure session renewal.
4	Session Management	express-session + connect-mongo provides persistent server-side sessions in MongoDB.
5	User Registration	New account creation with bcrypt password hashing and input validation.
6	User Login	Credential verification, JWT generation, and secure cookie delivery.
7	User Logout	Session destruction, token invalidation, and cookie clearing on logout.
8	Password Change	Authenticated users can update their password with old-password verification.
9	Role-Based Access Control	Admin and User roles with clearly defined permission boundaries.
10	Admin Dashboard	Protected admin panel for full platform management.

#	Feature	Description
11	Category Management	Admins create and manage unlimited categories with status control.
12	Topic Management	Admins create topics linked to categories for structured content tagging.
13	Rate Configuration	Configurable rate_per_1000 earning value per category, updateable at any time.
14	Category Status Control	Active/inactive toggle on categories controls writer visibility.
15	Dynamic Topic Rendering	Frontend JS reads backend data from HTML attributes to render topic lists dynamically.
16	Data via HTML Attributes	Backend data serialized as JSON into HTML data attributes for client-side consumption.
17	Content Moderation AI	AI API screens all user content for harmful/toxic material before DB writes.
18	Content Security Policy	Strict CSP headers prevent XSS and unauthorized script execution.
19	Secure Script Loading	External scripts restricted to trusted CDN sources only.
20	API Security	All API routes protected by auth middleware and input validation checks.
21	SEO Friendly Routing	Clean descriptive URLs improve SEO ranking and shareability.
22	Smart Redirect Logic	Automatic redirection based on authentication state.
23	Flash Messaging System	Session-based one-time messages for action confirmations and error feedback.
24	Navbar State Switching	Navbar UI adapts based on whether the current user is authenticated.
25	Middleware Architecture	Layered middleware pipeline for auth, validation, error catching, and sessions.
26	Async Error Handling	asyncHandler wrapper catches all async controller errors automatically.

#	Feature	Description
27	Database Modeling	Structured MongoDB collections with Mongoose schema validation.
28	User Model	Stores credentials, role, refresh token, and profile data.
29	Category Model	Stores category name, earning rate, and active/inactive status.
30	Topic Model	Stores topic name with categoryId foreign key reference.
31	Session Storage	MongoDB-backed session collection via connect-mongo adapter.
32	Backend Architecture	Modular MVC structure with controllers, routes, models, and utils.
33	Express Routing System	Route groups handle all API endpoints and EJS view rendering.
34	Form Handling	Server processes login, register, article, and admin forms.
35	Input Validation	Server-side validation for all required fields and data formats.
36	Error Handling System	Centralized error middleware with user-friendly messages.
37	Frontend Template Engine	EJS renders dynamic HTML server-side with injected controller data.
38	Layout Components	Reusable EJS partials for navbar, footer, and head sections.
39	Footer Component	Full footer with company links, legal pages, newsletter, and social icons.
40	Client-Side JavaScript	Browser JS handles dynamic rendering, dropdowns, and form interactivity.
41	JSON Data Rendering	Backend JSON data populates UI dynamically without extra API calls.
42	Environment Configuration	All secrets and config stored in .env, never in source code.

#	Feature	Description
43	Secure Configuration	Tokens, DB URIs, and session secrets stored exclusively in env vars.
44	Project Modularity	Code separated by concern for maintainability and ease of extension.
45	Scalable Backend Design	Architecture designed to add new features without breaking existing ones.

8.2 Feature Summary by Module

Module	Feature Numbers	Count
Authentication & User Mgmt	1–8: Auth, JWT, Refresh Tokens, Sessions, Register, Login, Logout, Password	8
Admin & Access Control	9–14: RBAC, Admin Dashboard, Categories, Topics, Rate Config, Status Control	6
Security	17–20: Content Moderation AI, CSP, Secure Scripts, API Security	4
Frontend & Dynamic Rendering	15–16, 21–24, 37–41: EJS, Dynamic, SEO, Flash, Navbar, Client JS	11
Middleware & Error Handling	25–26, 34–36: Pipeline, Async, Forms, Validation, Errors	5
Data Models & Database	27–33: DB Design, User Model, Category, Topic, Sessions, Architecture	7
Config & Scalability	42–45: Environment Config, Secure Config, Modularity, Scalable Design	4
TOTAL		45